Mochilas por fuerza bruta

Guion para grabar una clase de 60 minutos

ALGOED · Pásalo · Clase de recuperación / refuerzo
Tema central: selección binaria y Po, LAB1 2024-1, pregunta 1.
Resultado: el alumno representa una solución, enumera candidatos y verifica todas las restricciones en C++.

Esta es una planificación de **60 minutos de explicación, demostración y resolución**. No es una narración de una hora para leer de corrido. Las indicaciones entre corchetes son acciones para el docente. Las pausas propuestas son cortas y tienen una tarea concreta.

Antes de encender la grabación

- Abre `01 – Clase de mochilas.html`; funciona sin conexión. Usa las flechas para navegar y F para pantalla completa. Los simuladores se manejan con botones y selectores.
- Deja abiertos `mochila_binaria.cpp`, `po.cpp`, `datos1.txt` y `Codigo C++/LEEME.md` en el editor. Usa fuente grande, de al menos 22 px, y evita mostrar archivos personales.
- Compila ambos programas siguiendo el LEEME. Ejecuta una vez el caso original de Po para comprobar la ruta del archivo. Graba a 1080p y haz una prueba breve de audio.
- Ten visible este guion en otra ventana o dispositivo. El cronómetro de la presentación sirve para controlar el tiempo real; los horarios de las diapositivas indican el plan.
- Deja preparadas las estructuras y la lectura de archivos. Durante la clase escribe y explica la enumeración y la validación: no dediques el bloque central a transcribir todo el lector de datos.

Distribución de la hora

Diapositiva	Tiempo	Qué debe quedar claro
1	00:00–02:00	Qué resolveremos y cómo trabajaremos
2	02:00–05:00	Decisión, restricciones y objetivo
3	05:00–08:00	Enunciado de la mochila binaria
4	08:00–12:00	Cromosoma y conversión a base 2
5	12:00–15:00	Enumeración completa y mejor factible
6	15:00–18:00	Por qué un criterio voraz puede fallar
7	18:00–22:00	Implementación de la mochila
8	22:00–25:00	Pruebas, empates y capacidad sin llenar
9	25:00–28:00	Del sí/no a varios destinos

Diapositiva	Tiempo	Qué debe quedar claro
10	28:00–31:00	Contador en base 3
11	31:00–34:00	Reglas de Po
12	34:00–37:00	Lectura de tablas y estructuras
13	37:00–41:00	Prerrequisitos en la misma mochila
14	41:00–44:00	Un validador de asignaciones completas
15	44:00–48:00	Simular, romper y reparar una solución
16	48:00–52:00	Conectar enumeración y validación en C++
17	52:00–55:00	Ejecutar Po e interpretar otra solución válida
18	55:00–58:00	Pruebas que descubren errores
19	58:00–60:00	Ejercicio de salida y cierre

1 · 00:00–02:00 · Presentación

Di: “Hoy vamos a resolver dos ejercicios que comparten una idea: representar todas las decisiones posibles y revisarlas de forma ordenada. Empezaremos con una mochila pequeña, donde cada artículo entra o no entra. Después usaremos esa misma idea para el ejercicio de Po del laboratorio 2024-1: cada arma puede ir a una de varias mochilas o quedarse fuera.

Al terminar, quiero que puedas responder tres cosas: qué significa cada posición de tu arreglo, cuántos candidatos vas a generar y qué condiciones hacen válida una solución. Si esas tres partes están bien, el código sale mucho más claro. Vamos a trabajar en C++ con fuerza bruta iterativa, que es la estrategia pedida en esa pregunta del laboratorio.”

Haz: muestra los dos archivos de código y vuelve a la presentación. Explica que las soluciones y los datos quedan disponibles para practicar. Dedica los últimos 30 segundos a recordar `%` como residuo y `/` como división entera. No presupongas que el alumno ya domina máscaras de bits.

2 · 02:00–05:00 · Modelar antes de programar

Di: “Antes del primer `for`, separo tres cosas. La decisión es lo que puedo escoger. La restricción es lo que no puedo incumplir. El objetivo me dice cuándo una respuesta es mejor o cuándo puedo detenerme.

En la primera mochila, la decisión es elegir artículos; la restricción es no exceder una capacidad; el objetivo es aprovechar el mayor peso posible. En Po, la decisión es asignar un destino a cada arma; las restricciones incluyen tipo, requisitos y poder; el objetivo es encontrar alguna asignación que venza a todos los guerreros. No son objetivos intercambiables.

Esto nos evita un error frecuente: ver la palabra mochila y copiar un algoritmo sin volver a leer qué se pide.”

Haz durante 90 segundos: dibuja tres columnas —decisión, restricción y objetivo—. Completa la fila de la mochila y deja la de Po parcialmente vacía; la completarás en el minuto 31. Explica que en esta sesión no hay ganancias monetarias separadas del peso, ni se está desarrollando programación dinámica.

Pregunta (20 s): “Si encuentro una selección que cabe, ¿ya terminé?” **Respuesta:** no necesariamente; si se pide maximizar, otra selección puede aprovechar mejor la capacidad.

3 · 05:00–08:00 · Primera mochila

Di: “Tenemos cuatro artículos, de pesos 1, 2, 10 y 4, y una capacidad de 15. Cada artículo se puede usar como máximo una vez. Buscamos la mayor suma que no exceda 15. Un artículo con peso 4 no se puede partir, y el de peso 10 tampoco se puede repetir.

Una selección puede representarse con cuatro decisiones: uno si llevo el artículo, cero si no lo llevo. Si selecciono los pesos 1, 10 y 4, tengo una suma de 15 y una selección `1 0 1 1` en el orden de la tabla. Como ninguna respuesta factible puede superar la capacidad, esta selección alcanza el mejor valor posible.”

Haz: calcula a mano tres selecciones: todos los artículos suman 17 y no caben; 2 y 10 suman 12 y sí caben; 1, 10 y 4 suman 15. Pide que identifiquen la diferencia entre factibilidad y optimalidad.

Pregunta (30 s): “Si la capacidad baja a 9, ¿cuáles quedan?” **Respuesta:** 1, 2 y 4 suman 7. Guarda esta respuesta para probar el programa; no existe una selección que sume 8 o 9 con estos artículos.

4 · 08:00–12:00 · Simulador binario

Di: “Cuatro artículos, dos decisiones por artículo: tenemos dos por dos por dos por dos, es decir, 16 candidatos. Eso incluye no llevar nada. Vamos a usar un contador de 0 a 15 para generarlos.

En esta entrega fijamos una convención: el primer artículo aparece a la izquierda y el último cambia más rápido. Con cuatro posiciones, el número decimal 11 se escribe `1 0 1 1` en binario. Sus valores posicionales son 8, 4, 2 y 1, así que ocho más dos más uno da once. No confundan ese 11, que identifica el candidato, con el peso 15 que lleva la mochila.”

Haz durante 2 minutos: abre el simulador de la diapositiva 4. Mueve el código por 0, 1, 2, 3, 11 y 15. Lee en cada caso el artículo correspondiente y la suma. Para 11, escribe las divisiones: `11 % 2 = 1`, luego `5 % 2 = 1`, luego `2 % 2 = 0`, finalmente `1 % 2 = 1`. Coloca los residuos desde la última posición hacia la primera.

Pregunta (30 s): “¿Qué artículo elige el código 1?” **Respuesta:** el último, de peso 4. Algunos materiales antiguos usan el orden contrario; es válido, pero hay que mantener una única convención al interpretar la selección.

5 · 12:00–15:00 · Enumerar y conservar el mejor

Di: “Ahora ya sabemos generar un candidato. El siguiente paso es evaluarlo: sumo los pesos marcados con uno. Si la suma excede la capacidad, lo descarto. Si cabe y mejora el mejor peso que había encontrado, guardo su suma y copio su selección.

Hay dos arreglos con trabajos diferentes: el candidato cambia todo el tiempo y la mejor selección conserva la respuesta. Si solo guardo la suma, después no podré decir qué artículos la producen. Si guardo una referencia al arreglo que sigo modificando, puedo perder la solución. En esta versión copiamos los valores.”

Haz durante 90 segundos: recorre los códigos 0 a 3 y después salta a 10 y 11, dejando una tabla con `código / suma / cabe / mejora`. Explica que saltas en la demostración, pero el programa sí recorre todos los códigos. En 11 aparece peso 15.

Aclara: el programa entregado termina el recorrido completo para hacer visible la enumeración. En este problema concreto se podría detener al alcanzar exactamente la capacidad, porque ya alcanzó una cota superior. No hagas ese corte con la primera solución que simplemente cabe.

6 · 15:00–18:00 · El error del “más pesado primero”

Di: “Puede parecer que basta con ordenar de mayor a menor y meter el primer artículo que quepa. Probemos esa idea antes de confiar en ella. La capacidad ahora es 6 y los pesos son 4, 3 y 3. Si llevo el 4 primero, quedan dos unidades y ya no entra ningún 3. Termino con peso 4. Pero si dejo el 4 y llevo los dos artículos de peso 3, consigo 6.

Un ejemplo exitoso no demuestra que una estrategia funcione siempre. Un contraejemplo sí basta para demostrar que esa regla voraz no garantiza la mejor respuesta para este problema.”

Haz: dibuja ambas elecciones lado a lado. Señala que los dos artículos de peso 3 son objetos distintos; usar cada uno una vez respeta el enunciado. Escribe sus selecciones: `1 0 0` y `0 1 1`.

Pregunta (30 s): “¿Cuál es el objetivo que falló en la elección de 4?” **Respuesta:** maximizar el peso utilizado. La selección era válida, pero no óptima. Usa el minuto restante para que el alumno explique esa diferencia con sus propias palabras.

7 · 18:00–22:00 · Código de mochila

Di: “Vamos al archivo `mochila_binaria.cpp`. La lectura ya está preparada. Nuestra atención está en tres partes: contar selecciones, cargar el cromosoma y conservar la mejor selección.

Para contar, empiezo en uno y multiplico por dos una vez por artículo. Así trabajo con enteros. Para convertir, avanzo desde `n - 1` hasta cero y uso residuo y división. Finalmente recorro los candidatos con `codigo < total`: si hay 16 candidatos, los códigos van de 0 a 15.”

Haz durante 3 minutos: escribe o reconstruye `cargarCromosoma`; después explica el doble bucle de `resolverMochila`. Marca dónde se reinicia `peso = 0` para cada candidato. Subraya `peso <= capacidad && peso > mejorPeso`. Explica por qué la igualdad sí está permitida frente a la capacidad, y por qué no hace falta sustituir el mejor resultado por otro de igual peso.

Completa: la selección vacía inicia la respuesta con peso cero. Las sumas se guardan en `long long`. El límite de esta versión es 20 artículos: no prometas que enumerar sirve para cualquier tamaño. Su coste es O(n·2^n), porque revisa hasta n artículos por candidato.

8 · 22:00–25:00 · Ejecutar y probar

Haz: ejecuta los tres archivos `mochila-15.txt`, `mochila-9.txt` y `mochila-voraz.txt` siguiendo el LEEME. Antes de cada ejecución, da 15 segundos para anticipar el resultado.

Di: “Con capacidad 15 obtenemos `1 0 1 1`, peso 15 y espacio libre cero. Con capacidad 9 obtenemos `1 1 0 1`, peso 7 y espacio libre dos. Que queden dos unidades no significa que el algoritmo falló: ninguna combinación de estos artículos llena mejor la mochila. Con los pesos 4, 3 y 3 obtenemos peso 6, que corrige la decisión voraz.

Si todos los artículos pesan más que la capacidad, el resultado correcto es la selección vacía. Y si varias selecciones tienen el mismo mejor peso, imprimir cualquiera de ellas satisface el objetivo, salvo que el enunciado añada un criterio de desempate.”

Haz durante el último minuto: cambia la capacidad a cero y ejecuta. Relaciona la salida con la inicialización del mejor resultado. Vuelve al caso original antes de seguir.

9 · 25:00–28:00 · Varias mochilas

Di: “Hasta ahora cada artículo tenía dos destinos: fuera o dentro. Si hay dos mochilas, cada artículo tiene tres destinos: cero significa no usarlo; uno significa mochila uno; dos significa mochila dos. Por eso necesitamos base tres.

El número de posiciones sigue siendo la cantidad de artículos. La base depende de las opciones por artículo. Para G mochilas y A artículos tenemos G más uno opciones en cada una de A posiciones: `(G + 1)^A` asignaciones completas.

Una ventaja de esta representación es que un artículo no puede estar a la vez en dos mochilas: tiene una sola posición y esa posición guarda un solo destino.”

Haz: dibuja tres casillas para A, B y C. Completa `1 2 0` y traduce: A va a M1, B a M2 y C queda fuera. Dibuja después la idea equivocada de resolver las mochilas por separado y elegir A en ambas. La representación global evita esa reutilización.

10 · 28:00–31:00 · Contador en base 3

Haz: usa el simulador de la diapositiva 10 con tres artículos. Recorre los códigos 0, 1, 2, 3 y 15. Pide que anticipen el siguiente dígito antes de cambiarlo.

Di: “Después de `0 0 2` viene `0 1 0`: el último dígito vuelve a cero y avanza el anterior, igual que un contador decimal después del nueve. Con tres posiciones en base tres hay 27 asignaciones, numeradas de cero a 26. El código decimal 15 produce `1 2 0`: uno por nueve, más dos por tres, más cero.

Aquí solo estamos aprendiendo a representar destinos. Aún no hemos dicho si esa asignación cumple el enunciado de un problema. Generar una asignación y validarla son pasos diferentes.”

Pregunta (30 s): “¿Cuántos candidatos hay con tres mochilas y dos armas?” **Respuesta:** $4^2 = 16$. La opción cero también cuenta. Dedica el minuto restante a convertir manualmente un código en base cuatro con dos posiciones.

11 · 31:00–34:00 · Qué pide Po

Di: “Ahora abrimos la pregunta uno del LAB1 2024-1. Hay hasta tres guerreros y doce armas. Cada arma puede usarse una sola vez. Para cada guerrero debemos preparar una mochila con armas de tipos permitidos; si un arma tiene prerrequisitos, esos prerrequisitos deben viajar en la misma mochila. Finalmente, la suma de poder debe ser estrictamente mayor que la del guerrero.

En este ejercicio no hay una capacidad de peso. No tenemos que maximizar el poder ni utilizar todas las armas. Necesitamos encontrar alguna asignación que cumpla todo. Si el recorrido completo no encuentra ninguna, informamos que no existe solución.

El enunciado pide fuerza bruta. Nuestra solución enumera asignaciones completas con un contador y las valida.”

Haz: completa la fila de Po en la tabla de decisión, restricciones y objetivo. Resalta “estrictamente mayor”. Para un guerrero de poder 10, escribe que una mochila de poder 10 falla y una de 11 supera ese umbral.

12 · 34:00–37:00 · Leer los datos sin perderse

Haz: muestra la tabla de la diapositiva 12 y `datos1.txt` en paralelo. Identifica los tres guerreros: 120 admite tipo 2; 160 admite tipos 1 y 3; 80 admite tipo 3. Recorre solo tres armas al principio: Z, P y R.

Di: “Z tiene poder 60, tipo 3 y no requiere otra arma. P tiene poder 80, tipo 1 y requiere Z. R tiene poder 38, tipo 2 y no tiene requisitos. El identificador es la letra; no debemos confundir la letra con la posición en el arreglo.

Las estructuras guardan esos datos. `Guerrero` almacena el poder y los tipos aceptados; `Arma` almacena identificador, poder, tipo y requisitos. El lector admite datos variables dentro de los máximos del enunciado, así que la búsqueda no depende de que Z esté siempre en la primera posición.”

Haz durante 1 minuto: ubica esas estructuras en `po.cpp`. Muestra que las sumas usan `long long`, mientras las letras sirven para localizar las armas requeridas. No leas el parser completo línea por línea; explica su formato y dónde informa una entrada inválida.

13 · 37:00–41:00 · Los prerrequisitos son locales

Di: “La regla no dice que el requisito esté en cualquier mochila. Dice en la misma mochila. Si P va a la mochila dos, Z también debe tener destino dos. Si Z queda fuera, falla. Si Z va a la mochila tres, también falla. Tener ambas armas seleccionadas globalmente no es suficiente.

Y esto se revisa para cada arma elegida. Si hay una cadena de requisitos, al revisar todas las armas también verificamos los requisitos de las armas que fueron requeridas. No basta con comprobar solo el arma de mayor poder.”

Haz durante 2 minutos: dibuja `P → Z`, `D → R`, `I → R`, `K → Z` y `H → Z,E`. Revisa la solución del PDF: mochila 1 con R,D,I suma 125; mochila 2 con Z,P,F suma 197; mochila 3 con G,L suma 122. Verifica tipos y requisitos explícitamente.

Pregunta (30 s): “¿Podemos usar H en estos datos?” **Respuesta:** no. H es tipo 2 y requiere Z, de tipo 3. Ningún guerrero admite a la vez tipos 2 y 3, así que ambos no pueden compartir una mochila válida. Es una deducción de este conjunto de datos; no hay que programar una prohibición especial contra la letra H.

14 · 41:00–44:00 · Diseñar el validador

Di: “Nuestro validador recibe una asignación completa. Empieza con la suma de poder de cada mochila en cero. Por cada arma, si su destino es cero, la omite. Si está asignada, comprueba el tipo y comprueba que cada requisito tenga exactamente ese mismo destino. Después incorpora su poder a la mochila correspondiente.

Al terminar, verifica todos los guerreros: si alguna suma es menor o igual al poder del guerrero, devuelve falso. Solo devuelve verdadero cuando todas las condiciones se cumplen.

La exclusividad ya viene garantizada por un destino por arma. Los requisitos y los tipos todavía deben comprobarse. La representación resuelve una parte del problema; no reemplaza al resto del validador.”

Haz: escribe el pseudocódigo de la diapositiva 14 en la pizarra. Traza una arma sin requisito, otra con requisito correcto y otra con requisito en una mochila diferente. Aclara que el orden de comprobar tipo y requisito puede variar sin cambiar el significado del resultado.

15 · 44:00–48:00 · Simular, romper y reparar

Haz con el simulador de Po:

1. Pulsa **Solución del enunciado**. Comprueba los tres poderes: 125, 197 y 122. Lee por qué P, D e I cumplen sus dependencias.
2. Cambia Z de M2 a “Sin usar”. P pierde su requisito y la mochila 2 también pierde poder. Pide que identifiquen ambos fallos; después restaura la solución.
3. Coloca H en M1. Tiene tipo compatible con el guerrero 1, pero sus requisitos Z y E no están allí. Lleva E a M1 y luego intenta llevar Z: al arreglar un requisito aparece un tipo incompatible. Explica por qué este caso no se arregla moviendo solamente H.
4. Pulsa **Primera solución del programa**. Compara la selección sin exigir que coincida con la del PDF.

Di: “Este simulador comprueba la asignación que estamos editando. No está recorriendo millones de candidatos en el navegador. El recorrido exhaustivo lo hace el programa C++. Aquí lo usamos para ver con claridad qué regla falla y para defender por qué una respuesta sí es válida.”

Reserva el último minuto: pide que propongan un cambio que rompa únicamente un tipo permitido o un requisito. Compruébalo en pantalla y vuelve a una solución válida antes de salir.

16 · 48:00–52:00 · Conectar el programa

Haz en `po.cpp` **durante 3 minutos:** muestra `contarAsignaciones`, `cargarCromosoma`, `encontrarSolucionFuerzaBruta` y `esAsignacionValida`. Marca la cadena de llamadas: el bucle genera, el validador responde y el buscador copia la primera respuesta válida.

Di: “En el caso máximo son 4^{12}, es decir, 16 777 216 asignaciones posibles. El bucle empieza en cero y usa menor que el total. La conversión llena todas las posiciones del cromosoma, por lo que no arrastra destinos de la iteración anterior. El validador reinicia las sumas cada vez que lo llamamos.

En cuanto encontramos una asignación válida, copiamos sus destinos y terminamos. Este corte sí es correcto porque el problema pide existencia. Si el enunciado pidiera el mayor poder o el menor número de armas, tendríamos que comparar las soluciones y cambiar el criterio de parada.”

Detalle para una pregunta de complejidad: la enumeración tiene `(G+1)^A` candidatos. Este validador busca cada requisito recorriendo hasta A armas; con un máximo fijo de tres requisitos por arma, el coste peor del recorrido completo es $O((G+1)^A \cdot A^2)$, más las comprobaciones de guerreros. No lo presentes como un algoritmo eficiente para cientos de armas.

17 · 52:00–55:00 · Correr el caso real

Haz: ejecuta `./po datos1.txt`. Si el entorno tarda, aprovecha para explicar que el número total de candidatos es una cota de lo que podría revisar, no una promesa sobre la velocidad de cualquier equipo.

Di: “Con el orden de lectura y la enumeración de esta entrega, la primera solución que imprime es R,I,J para el guerrero uno, con poder 142; Z,P,K para el dos, con 176; y G,L para el tres, con 122. El programa probó 10 751 068 candidatos antes de encontrarla.

Esta solución es distinta de la del ejemplo, pero satisface la misma especificación. I tiene a R en su mochila; P y K tienen a Z; todos los tipos están permitidos y todos los poderes superan sus umbrales. Ninguna arma aparece en dos mochilas.”

Haz durante 90 segundos: subraya cada requisito satisfecho en la salida. Compara con el PDF y aclara que no se deben codificar las letras de esa respuesta como si fueran una solución general. El algoritmo debe volver a buscar si cambian los datos.

18 · 55:00–58:00 · Pruebas que sí sirven

Haz: ejecuta los casos cortos de Po. Da al alumno 15 segundos para predecir cada resultado.

Di: “Primero, guerrero de poder 10 y una única arma de poder 10 del tipo correcto. No hay solución: igualdad no es victoria. Segundo, un arma A de poder 6 que requiere B de poder 5; ambas son de tipo permitido y van juntas. Aquí sí hay solución, con poder 11. Tercero, arma de poder 20 frente a guerrero de poder 10, pero el tipo no es admitido. Tampoco hay solución: tener poder suficiente no elimina las otras restricciones.

Una prueba útil intenta descubrir una equivocación concreta. Si todas mis pruebas solo contienen soluciones válidas, podría no detectar que mi código acepta armas incompatibles. También debo probar que un arma no pueda reutilizarse para vencer a dos guerreros.”

Haz durante el último minuto: vuelve a la condición `suma_poder[g] <= guerreros[g].poder`. Explica cómo cambiar `<=` por `<` haría fallar la primera prueba. No alteres permanentemente el archivo entregado; basta señalar la mutación en la pizarra.

19 · 58:00–60:00 · Comprobación final

Di: “Último ejercicio: hay dos guerreros y tres armas. ¿En qué base representamos la asignación y cuántos candidatos hay?”

Pausa activa (20 s). Respuesta: base 3; $3^3 = 27$. Pide interpretar `2 0 1`: A va a M2, B queda fuera y C va a M1.

Pregunta (20 s): “¿Cuándo puedo detener el recorrido?” **Respuesta:** en Po, con la primera solución válida; en la mochila de máximo peso, cuando acabe de comparar todas o cuando alcance una cota superior demostrada, como la capacidad exacta de este caso.

Cierra: “Para practicar, repite la mochila con otra capacidad y después modifica un requisito de Po. Antes de ejecutar, predice si tu cambio debería permitir o impedir una solución. Tienes la presentación, los casos y el código para comprobarlo. Lo esencial es mantener conectadas estas tres ideas: representar todas las decisiones, validar el enunciado completo y detenerse según el objetivo correcto.”

Si necesitas ajustar el tiempo durante la toma

- **Si vas 2 minutos adelantado:** usa la práctica de transferencia: dos mochilas de capacidades 5 y 4, artículos 4,3,2. Codifica `2 1 1` en base 3 y verifica que las cargas sean 4 y 5 en las mochilas 2 y 1, respectivamente. No añadas otro tema.
- **Si vas 2 minutos atrasado:** resume la lectura de estructuras, que ya está preparada, y reduce la escritura en vivo. Conserva la explicación de requisitos, poder estricto y las pruebas finales.
- **Para editar después:** graba transiciones breves entre bloques. Retira esperas técnicas largas y errores de apertura de archivos; conserva el razonamiento y las pausas activas. La duración final depende de la toma y del montaje.

Procedencia y alcance

La mochila inicial adapta el ejemplo del banco local de ALGOED. El problema de Po, sus tablas y condiciones proceden del enunciado PUCP LAB1 2024-1, pregunta 1, incluido en los materiales. Los ejercicios de transferencia y esta secuencia docente son material de apoyo preparado para la grabación; no representan una solución oficial de la universidad. No se presenta esta entrega como una grabación de una sesión anterior.